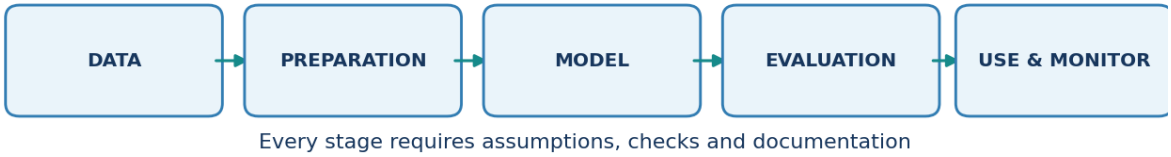
פרק 8 — מושגי יסוד בלמידת מכונה

ממודל שמחשב תחזית למערכת שלומדת, נבדקת ומשתפרת

ספר תלמיד | התמחות למידת מכונה ולמידה עמוקה — 5 יחידות לימוד | 10 שעות על פי מפת התכנית שבמאגר.

מעמד הפרק: פרק 8 בתכנית ההתמחות. דוגמאות על Gradient Descent ורגולריזציה מובאות כהכנה לפרקי הפרספטרון והמתמטיקה של למידה עמוקה; אין לייחס להן מעמד של תת־סעיף חובה בפרק זה ללא הצלבת נוסח הסילבוס הרשמי המלא.



איור: מסלול פיתוח מודל: עבודה עם נתונים, הכנה, התאמת מודל, הערכה ושימוש אחראי לאורך זמן.

איך לומדים בעזרת הספר?

בכל מושג נתחיל משאלה מחיי היום־יום, נציג דוגמה שניתן לחשב ביד, ננסח את העיקרון המתמטי, נבדוק אותו ב־Python ולבסוף ננתח מקרה שבו התוצאה עלולה להטעות. אין צורך לזכור את שמות הפונקציות בעל פה: החשוב הוא להבין מה כל פעולה מקבלת, מה היא מחזירה ומדוע בחרנו לבצע אותה.

מטרות ביצועיות

- להבחין בין נתונים, מאפיינים, משתנה מטרה, תחזית, פרמטר והיפר־פרמטר.
- להסביר כיצד משתמשים בפונקציית שגיאה כדי להשוות תחזיות, ולחשב MAE ו־MSE ביד.
- להבין את רעיון שינוי הפרמטרים בעת אימון, כולל דוגמת צעד אחת של Gradient Descent כהכנה.
- לזהות Overfitting ו־Underfitting בעזרת גרפים והבדלים בין אימון לוולידציה.
- לתכנן ניסוי הוגן עם חלוקת נתונים, קו בסיס ומניעת זליגה.
- לנסח החלטה מקצועית מנומקת מתוך מדדים, מגבלות ויומן ניסויים.

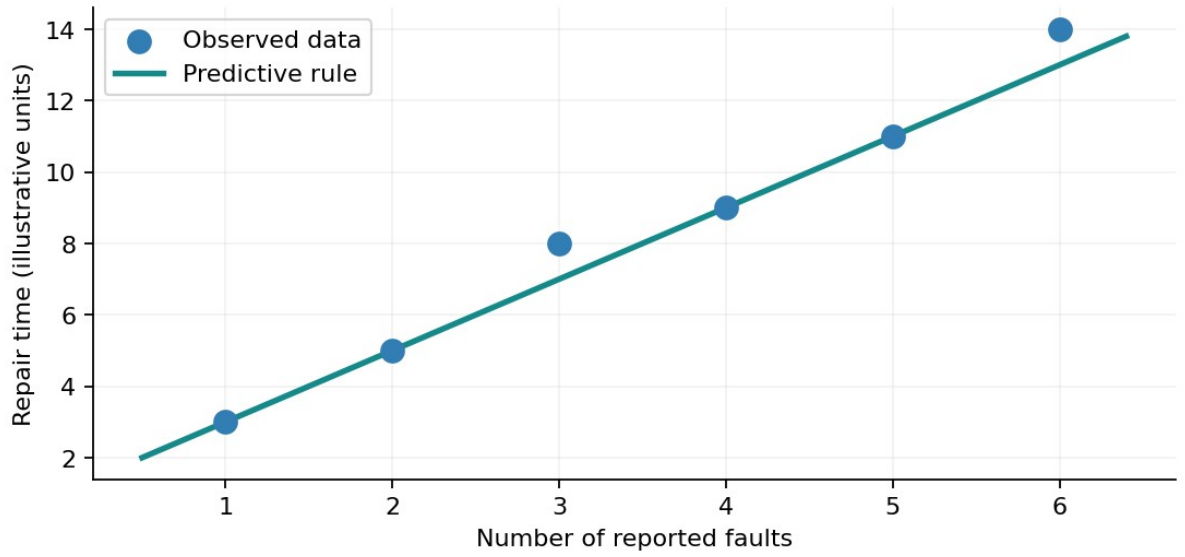
ידע קודם

הקורא כבר למד פונקציות בסיסיות, וקטורים, טבלאות וגרפים, רגרסיה ליניארית, KNN, מדדי הערכה ו־Train/Validation/Test. אין צורך ללמוד אותם מחדש. במקום זאת נשתמש בהם כדי להבין את תהליך האימון והבחירה במודל ברמה גבוהה יותר.

1. סיפור פתיחה: מחשב שחוזה זמן תיקון — האם הוא באמת לומד?

דודו אחראי על מעבדת המחשבים בבית הספר. לאחר כל תיקון הוא רושם כמה תקלות אותרו וכמה זמן ארך התיקון. הוא רוצה לבנות מערכת שתעריך מראש את משך הטיפול במחשב חדש, כדי לתכנן את עבודת הטכנאים. בתחילה הוא כותב כלל קבוע: ״כל תקלה מוסיפה עשר דקות״. אבל בבדיקת נתוני עבר הוא מגלה שהכלל מתאים לחלק מהמקרים ולאחרים פחות.

המעבר האמיתי ללמידת מכונה אינו עצם כתיבת נוסחה. בלמידת מכונה אנחנו בוחרים משפחה של כללים אפשריים, משתמשים בנתונים כדי להתאים את הפרמטרים של הכלל, ובודקים האם הכלל שנבחר עוזר לנו לחזות מקרים שלא שימשו להתאמתו.



איור: דוגמה לימודית של מספר תקלות מול זמן תיקון. הנקודות הן מדידות; הקו הוא כלל חיזוי. הוא אינו חייב לעבור בכל נקודה.

1.1 מה אנחנו יודעים ומה המודל צריך ללמוד?

תפקיד	דוגמה במעבדה	מרכיב
דוגמה אחת ללמידה	מחשב שטופל בעבר	תצפית
מידע ידוע בזמן החיזוי	מספר התקלות	מאפיין x
מה שרוצים לחזות	משך התיקון בפועל	מטרה y
תוצאת המודל	משך תיקון משוער	תחזית ŷ
ערכים המותאמים לנתונים	שיפוע וחיתוך	פרמטרים

שאלת הבנה: האם "משך התיקון בפועל" הוא מאפיין שמותר להזין למודל כאשר הוא מנבא משך תיקון שטרם התרחש? לא. זהו בדיוק המשתנה שאנחנו מנסים לחזות. הכנסת התוצאה האמיתית לתכונות בזמן האימון תייצר זליגת מידע חמורה.

2. מהו מודל וכיצד הוא שונה מאלגוריתם האימון?

מודל הוא הפונקציה שמקבלת נתונים ומפיקה תחזית. אלגוריתם האימון הוא התהליך שמחפש ערכי פרמטרים מתאימים. בסיום האימון שומרים את המודל המותאם, ולא את כל סדר השאלות והטעויות שביצענו בדרך. לדוגמה, כאשר עובדים עם רגרסיה ליניארית, המודל הוא ישר; שיטת התאמת המקדמים עשויה להיות חישוב מתמטי ישיר או תהליך איטרטיבי.

$$\hat{y} = wx + b$$

במשוואה זו x הוא מאפיין הקלט, w הוא המקדם שמכפיל אותו, b הוא החיתוך עם ציר y, ו־ŷ היא התחזית. שים לב להבדל בין y, הערך האמיתי, לבין ŷ, הניחוש של המודל. ההבדל ביניהם עומד בבסיס פונקציית השגיאה.

2.1 פרמטר לעומת היפר־פרמטר

פרמטר הוא ערך שנקבע מתוך נתוני האימון בהתאם לאלגוריתם. היפר־פרמטר הוא בחירה שמגדירה כיצד ייבנה או יופעל תהליך הלמידה. הוא עשוי להיבחר בידי המפתח או באמצעות חיפוש על נתוני ולידציה. חשוב שלא לבלבל היפר־פרמטר עם תכונה של דוגמה אחת.



איור: הבחנה בין ערכים שהמודל מתאים מנתונים לבין הגדרות תהליך שהמפתח בוחר או מכוון.

מודל	פרמטר שנלמד	דוגמה להיפר־פרמטר
רגרסיה ליניארית	משקולות w והטיה b	מידת רגולריזציה ב־Ridge
KNN	אין משקולות גלובליות מהסוג הזה; דוגמאות האימון נשמרות	מספר השכנים k
רשת עצבית	משקולות והטיות של הנוירונים	קצב למידה, מספר שכבות

שאלת חשיבה: נניח שבחרנו ב־KNN עם k=5. האם המודל "למד" את המספר 5? לא. זהו ערך שנקבע בתהליך תכנון או כוונון. לעומת זאת, ברשת עצבית המשקולות בדרך כלל משתנות תוך כדי האימון.

3. איך מודדים עד כמה התחזית טובה? פונקציית השגיאה

כדי לשפר מודל דרושה דרך כמותית לתאר עד כמה הוא טועה. לא מספיק לומר שהתחזית "קרובה" לערך האמיתי, משום שקשה להשוות בין מאות תחזיות ללא כלל חישוב משותף. פונקציית שגיאה (Loss) הופכת את הפער בין התחזית לערך האמיתי למספר. כשמקבלים מספר כזה אפשר להשוות בין מודלים או בין שלבי אימון שונים.

3.1 שארית — הצעד הקטן הראשון

$$e_i = y_i - \hat{y}_i$$

אם זמן התיקון בפועל הוא 8 יחידות זמן והמודל חזה 7, השארית היא 1. אם הוא חזה 9, השארית היא −1. הסימן מספר לנו לאיזה כיוון טעה המודל, אבל כאשר מסכמים שאריות חיוביות ושליליות הן עלולות לבטל זו את זו. לכן משתמשים לעיתים בערך מוחלט או בריבוע של הטעות.

3.2 MAE — ממוצע השגיאות המוחלטות

$$\mathrm{MAE} = \frac{1}{n}\sum |y_i - \hat{y}_i|$$

MAE מודד בכמה יחידות, בממוצע, התחזית מתרחקת מן התוצאה האמיתית. היתרון הדידקטי שלו הוא שהיחידות נשמרות: אם אנחנו חוזים דקות, גם MAE נמדד בדקות. מדד זה נוח להסבר לאנשים שאינם מפתחים, אך הוא אינו מעניש שגיאות גדולות באותה עוצמה כמו ריבוע השגיאה.

3.3 MSE — ממוצע ריבועי השגיאות

$$\mathbf{MSE = (1/n)\ \Sigma\ (y_i - \hat{y}_i)^2}$$

MSE נותן משקל גדול יותר לשגיאות גדולות כי הוא מעלה כל שגיאה בריבוע. הוא שכיח מאוד בפונקציות אובדן לאימון מודלי רגרסיה, משום שהוא נוח לניתוח מתמטי. אולם היחידות שלו הן יחידות המדידה בריבוע, ולכן פירוש גודל התוצאה פחות מיידי. שורש ה־MSE, כלומר RMSE, מחזיר אותנו ליחידות המקור.

4. חישוב ידני מלא: איזה מודל עדיף?

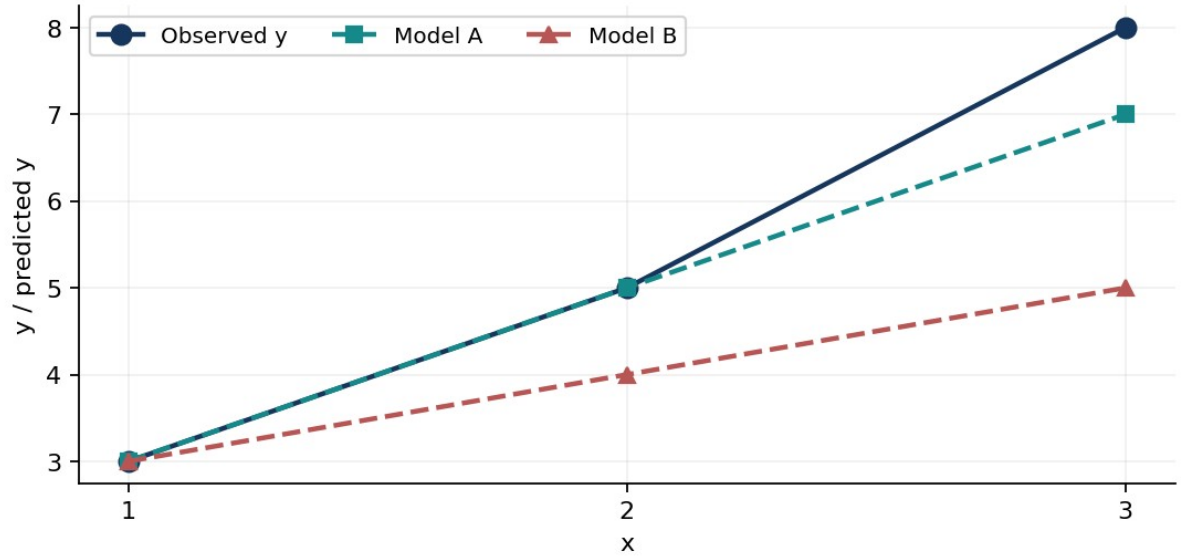
נשתמש בשלוש דוגמאות פשוטות. ערכי הקלט הם 1, 2, 3; ערכי המטרה הם 3, 5, 8. ננסה שני כללים: מודל A עם שיפוע 2 וחיתוך 1, ומודל B עם שיפוע 1 וחיתוך 2. נחשב תחזית לכל תצפית ולאחר מכן את השארית ואת ריבועה.

x	ערך אמיתי y	תחזית A: 2x+1	$(y-\hat{y}A)^2$	תחזית B: x+2	$(y-\hat{y}B)^2$
1	3	3	0	3	0
2	5	5	0	4	1
3	8	7	1	5	9
סכום			1		10

$$\mathbf{MSE(A) = (0 + 0 + 1) / 3 = 1/3 \approx 0.3333}$$

$$\mathbf{MSE(B) = (0 + 1 + 9) / 3 = 10/3 \approx 3.3333}$$

לכן על שלוש הדוגמאות האלה מודל A השיג MSE קטן יותר. אבל איננו יודעים מכאן שהוא יהיה טוב יותר לכל מחשב חדש. ייתכן שהנתונים קטנים מדי, וייתכן שהדוגמאות נבחרו באופן שאינו מייצג את האוכלוסייה. הבחנה זו תקפה לכל מדד: המדד מתאר את הביצועים על הנתונים שעליהם חושב.



איור: השוואת התחזיות של שני מודלים לאותן שלוש תצפיות. הפער בין קו לנקודת אמת הוא השגיאה.

```
import numpy as np
x = np.array([1, 2, 3])
y = np.array([3, 5, 8])
pred_a = 2 * x + 1
pred_b = x + 2
mse_a = np.mean((y - pred_a) ** 2)
mse_b = np.mean((y - pred_b) ** 2)
print("A:", mse_a)
print("B:", mse_b)
```

לפני ההרצה, נסו לחזות את המספרים שיתקבלו. לאחר מכן השוו בין הפלט לחישוב הידני. אם התוצאות אינן זהות, אל תחליפו מיד את החישוב: בדקו אילו ערכים נכנסו למערך ובאיזה שלב.

5. מה פירוש "אימון"? התאמת הפרמטרים

לאחר שבחרנו דרך למדוד שגיאה, אפשר לנסות לשנות את פרמטרי המודל כך שהשגיאה תקטן. יש אלגוריתמים שמוצאים את הפתרון באמצעות נוסחה; אחרים מתחילים מערכים כלשהם ומעדכנים אותם בהדרגה. לכן המילה "אימון" אינה מציינת בהכרח אותה שיטת חישוב בכל המודלים.

ברגרסיה ליניארית אפשר למצוא מקדמים באמצעות שיטת ריבועים פחותים. בהכנה ללמידה עמוקה חשוב להבין גם את הרעיון האיטרטיבי: מתחילים מניחוש, מחשבים שגיאה, בודקים באיזה כיוון שינוי הפרמטרים צפוי להקטין את השגיאה ומבצעים צעד קטן. ברשתות עצביות התהליך יהיה עשיר יותר ונלמד אותו בהמשך.

5.1 הדגמת צעד אחד של Gradient Descent — הרחבת גשר

נתחיל בדוגמה עם נתון יחיד כדי להתמקד במשמעות הפעולה. נניח שמודל פשוט מנבא $\hat{y}=wx$, כאשר $x=2$ והערך האמיתי $y=5$. נבחר תחילה $w=1$ ונחשב את השגיאה הריבועית.

$$L(w) = (2w - 5)^2$$

$$L(1) = (2 - 5)^2 = 9$$

כדי לדעת כיצד לשנות את המשקל נבחן את נגזרת פונקציית השגיאה. אין צורך בשלב זה לגזור פונקציות רבות; נשתמש בתוצאה ונפרש אותה. בפרק המתמטיקה של הלמידה העמוקה נלמד את הגזירה בצורה שיטתית.

$$dL/dw = 4(2w - 5)$$

$$dL/dw \mid (w=1) = -12$$

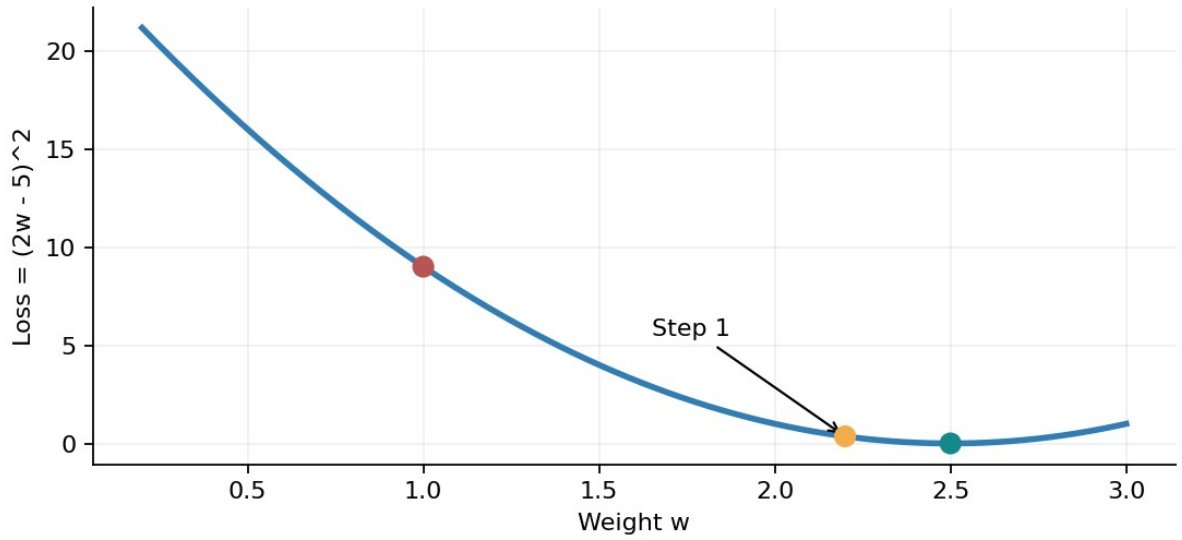
הנגזרת שלילית בנקודה זו. פירוש הדבר: אם נגדיל מעט את w, השגיאה אמורה לקטון. כלל העדכון מחסר מן המשקל את קצב הלמידה כפול הנגזרת; במקרה של נגזרת שלילית, החיסור גורם להגדלת המשקל.

$$w_{new} = w_{old} - \eta \, (dL/dw)$$

$$w_{new} = 1 - 0.1 \times (-12) = 2.2$$

$$L(2.2) = (4.4 - 5)^2 = 0.36$$

השגיאה ירדה מ־9 ל־0.36. זו דוגמה אחת בלבד, ולא הבטחה שכל גודל צעד ישפר תמיד את התוצאה. אם קצב הלמידה גדול מדי, נוכל לדלג מעל המינימום ואף להגדיל את השגיאה. זהו בדיוק אחד הנושאים שנחקור בהמשך הלמידה העמוקה.



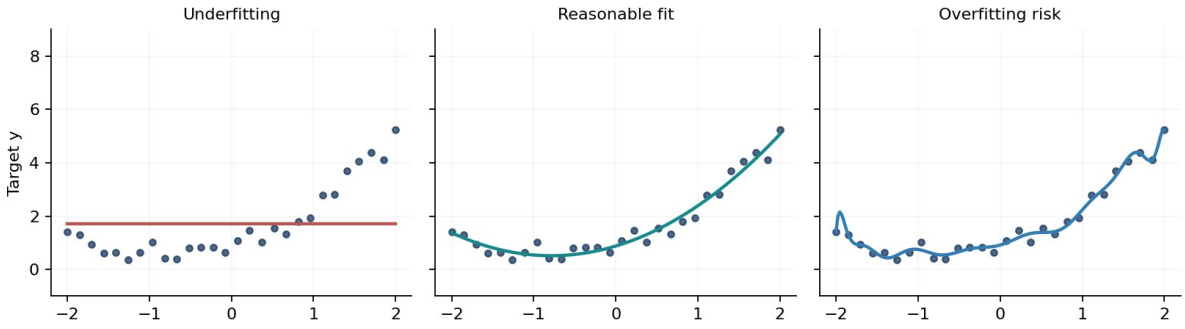
איור: פונקציית שגיאה כפונקציה של משקל יחיד. צעד לעבר המינימום עשוי להקטין את השגיאה.

```
w = 1.0
x = 2.0
y = 5.0
learning_rate = 0.1
loss_before = (w * x - y) ** 2
gradient = 2 * (w * x - y) * x
w = w - learning_rate * gradient
loss_after = (w * x - y) ** 2
print(w, loss_before, loss_after)
```

בדיקת הבנה: מה ישתנה אם קצב הלמידה יהיה 1 במקום 0.1? חשבו את הצעד ביד לפני שתריצו את הקוד, ונסו להסביר את התוצאה בעזרת גרף הפונקציה.

6. הצלחה באימון אינה הצלחה בעולם האמיתי

מודל עשוי לשנן דפוסים שמופיעים בנתוני האימון, כולל רעש, מקרים חריגים או פרטים ייחודיים שאינם חוזרים באוכלוסייה. אם הוא מצליח מאוד על נתוני האימון אבל נכשל על דוגמאות חדשות, יש חשד להתאמת יתר — Overfitting. מצב הפוך, שבו המודל פשוט מדי ואינו מזהה אפילו את הדפוס הבסיסי בנתונים, נקרא Underfitting.

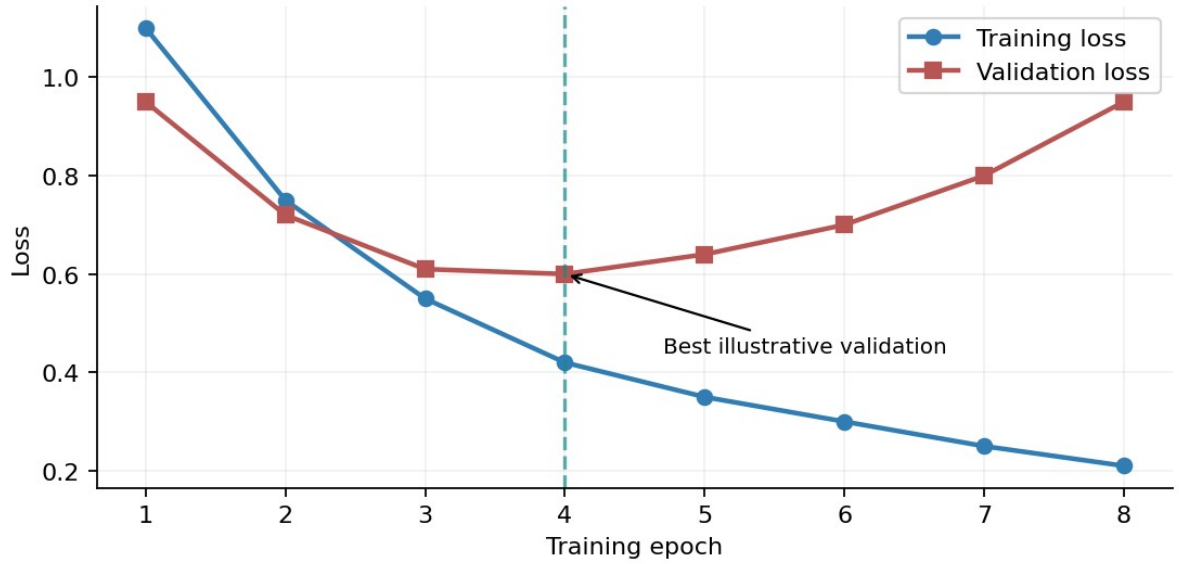


איור: שלוש דרגות התאמה: מודל פשוט מדי, מודל סביר ומודל מורכב מאוד שיכול לעקוב אחרי רעש המדגם.

חשוב להיזהר מהכללה: מודל מורכב אינו בהכרח מודל גרוע, ומודל פשוט אינו בהכרח טוב. איכות ההתאמה תלויה במורכבות הבעיה, בכמות הנתונים, ברמת הרעש ובאופן ההערכה. ההחלטה צריכה להסתמך על נתונים שאינם משמשים לבחירת הפרמטרים.

6.1 כיצד קוראים עקומות אימון?

במודלים מאומנים בתהליך איטרטיבי אפשר לצייר את השגיאה לאחר כל תקופת אימון (Epoch). כאשר שגיאת האימון יורדת ושגיאת הוולידציה יורדת יחד איתה, מתקבלת עדות לכך שהתהליך משפר את הכללת המודל. כאשר שגיאת האימון ממשיכה לרדת אך שגיאת הוולידציה מתחילה לעלות באופן עקבי, ייתכן שהמודל מתאים את עצמו לפרטי האימון מעבר לרצוי.



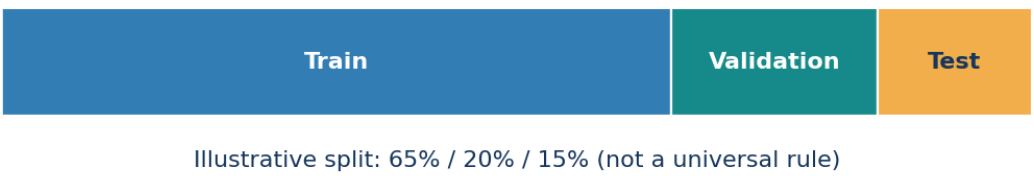
איור: עקומות אימון לדוגמה בלבד: לאחר תקופת אימון מסוימת, שגיאת הוולידציה עולה אף ששגיאת האימון יורדת.

הערה חשובה: עקומת אימון חריגה אחת אינה הוכחה מתמטית ל־Overfitting. יש לבחון כיצד נוצרו הקבוצות, האם חל שינוי בהתפלגות, האם ההבדלים יציבים בניסויים חוזרים והאם המדד מתאים לבעיה.

איזו בדיקה נבצע?	מה אפשר לחשוד?	דפוס
בדיקת מאפיינים, איכות נתונים ומורכבות	Underfitting או בעיית נתונים	שגיאת אימון גבוהה ושגיאת ולידציה גבוהה
בדיקה עצמאית, רגולריזציה או יותר נתונים	Overfitting אפשרי	אימון נמוך מאוד, ולידציה גבוהה
בדיקת Test חיצונית ובחינת מגבלות	הכללה טובה על הדגימה שנבדקה	אימון וולידציה דומים ונמוכים

7. מדוע מחלקים נתונים? אימון, בחירה ובדיקה

בפרק 5.2 של נושאי הליבה הכרנו Train, Validation ו־Test ואת K-Fold. כאן נשתמש בידע כדי לתכנן מערכת שלמה. קבוצת Train מאפשרת להתאים פרמטרים; Validation מסייעת לבחור מודל והיפר־פרמטרים; Test נועדה לתת הערכה סופית לאחר שהבחירות כבר התקבלו. אם נשתמש ב־Test שוב ושוב כדי לבחור מודלים, היא כבר לא תהיה מבחן עצמאי אמיתי.

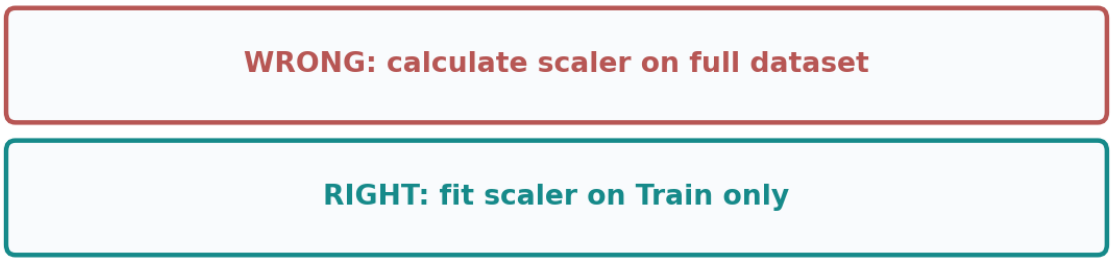


איור: חלוקת נתונים לדוגמה בלבד. אחוז החלוקה תלוי בכמות הנתונים, באיזון ובמבנה הזמן או הקבוצות.

אם יש כמה רשומות מאותו מחשב, צריך לשקול חלוקה לפי מזהה מחשב כדי שלא יהיו רשומות כמעט זהות גם באימון וגם בבדיקה. כאשר המטרה היא לחזות אירועים בעתיד, לעיתים נדרשת חלוקה לפי זמן: עבר לאימון ותקופה מאוחרת לבדיקה. חלוקה אקראית עלולה להיות בלתי מתאימה במקרה זה.

7.1 זליגת מידע — טעות שמציגה מודל מוצלח מדי

זליגת מידע מתרחשת כאשר מידע שלא היה זמין במועד החיזוי משפיע על בניית המודל או על הערכתו. לדוגמה, אם נשתמש במשך התיקון בפועל כאחד המאפיינים של המחשב, המודל יקבל במישרין או בעקיפין את התשובה. גם חישוב ממוצע וסטיית תקן על כל המאגר לפני חלוקה ל־Train/Test עלול לשלב מידע מן הבדיקה בתהליך האימון.



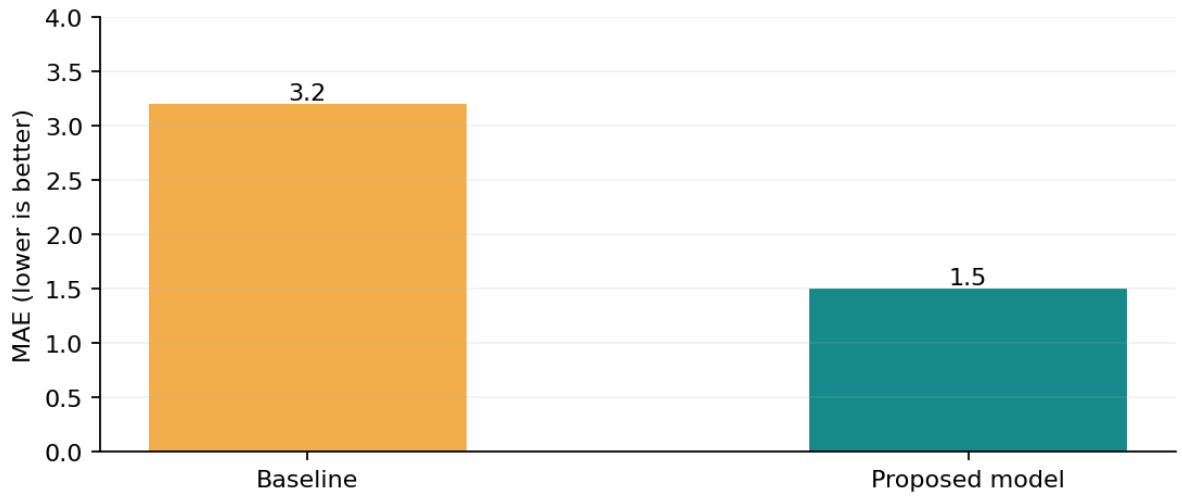
איור: התאמת פעולות עיבוד צריכה להישען על נתוני האימון בלבד; הנתונים האחרים עוברים באמצעות *transform* ההתאמה הקיימת.

הפתרון המקצועי הוא להפריד תחילה קבוצות ולהכניס פעולות עיבוד נלמדות ל־Pipeline. בתוך Cross-Validation הפעולות נלמדות מחדש על חלק האימון של כל קיפול. כך תוצאות ההערכה קרובות יותר למצב שבו המודל יראה דוגמה חדשה באמת.

8. מהו קו בסיס (Baseline) ומדוע הוא הכרחי?

מודל שמקבל ציון מסוים יכול להיראות מרשים, אך ללא השוואה אין לנו נקודת ייחוס. בבעיית רגרסיה אפשר לבנות קו בסיס שמנבא תמיד את ממוצע ערכי המטרה באימון. בבעיית סיווג אפשר לנבא תמיד את המחלקה הנפוצה ביותר, או להשתמש בכלל עסקי פשוט. קו הבסיס אינו אמור להיות חכם; הוא אמור להגדיר את הרף שהמודל החדש צריך להתעלות עליו.

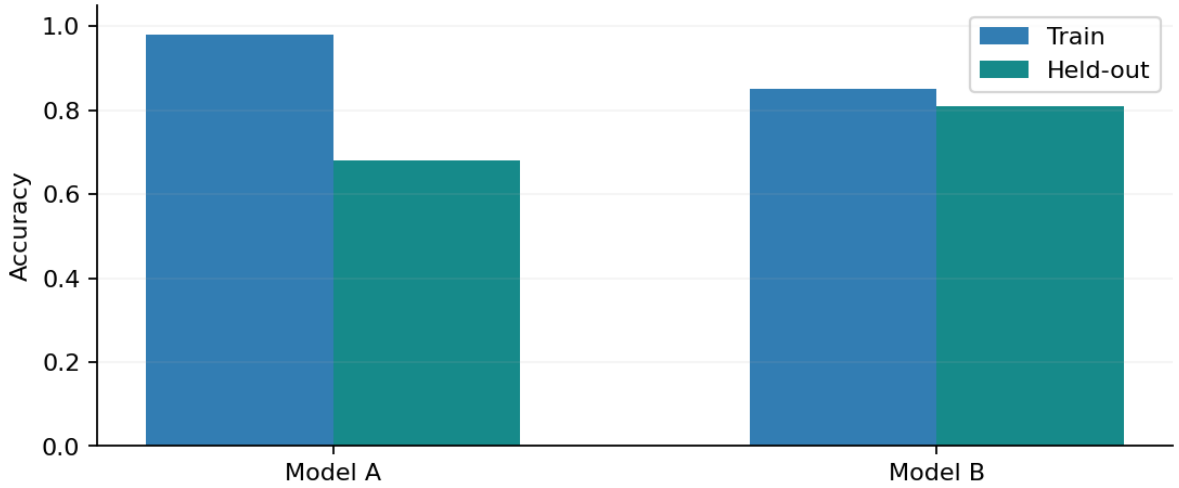
נניח ש־MAE של קו הבסיס הוא 3.2 יחידות זמן, ושל המודל שלנו הוא 1.5. זו ראיה לכך שהמודל עדיף לפי המדד ועל מערך הבדיקה שנבחר. אבל אם מודל חדש מסוכן יותר או יקר יותר להפעלה, השיפור המספרי לבדו אינו מבטיח שזו הבחירה הנכונה.



איור: השוואת *MAE* לדוגמה: תוצאת מודל חייבת להיבחן ביחס לקו בסיס ולמגבלות המדידה.

8.1 מקרה השוואה: דיוק אימון מול יכולת הכללה

לשני מודלים יש ציוני אימון ובדיקה שונים. מודל A משיג Accuracy של 98% באימון אך רק 68% בנתוני בדיקה חיצוניים; מודל B משיג 85% באימון ו־81% בבדיקה. אם שניהם נבחנו על אותן דוגמאות ובאותם תנאים, מודל B עדיף לפי Accuracy של הבדיקה. אך לפני שמכריעים סופית נבדוק אילו טעויות כל מודל עושה ואיזה מחיר יש לכל טעות.



איור: דוגמה להבחנה בין הצלחה באימון לביצועי הכללה. הנתונים באיור מומצאים לצורכי הוראה.

9. ניסוי מדעי במודל: איך משפרים בלי להטעות את עצמנו?

חקר מודלים אינו תחרות של הרצת כמה שיותר אלגוריתמים. ניסוי טוב מתחיל בהשערה ברורה: "אולי קנה המידה גורם לכך שמאפיין אחד משתלט על מרחקי KNN". נשנה גורם אחד, נמדוד תוצאה על אותם קיפולים או אותה קבוצת ולידציה, נתעד את התוצאה ונשקול הסברים חלופיים. שינוי כמה גורמים בבת אחת יקשה עלינו לדעת מה השפיע.



איור: מחזור ניסוי: השערה, שינוי מוגדר, מדידה, תיעוד והסבר מגבלות.

ניסוי	מה משנים?	מה מחזיקים קבוע?	מה מודדים?
1 — קו בסיס	סוג המודל	חלוקת הנתונים	MAE ומשך אימון
2 — תקנון	עם/בלי StandardScaler	אותו מודל ואותם קיפולים	MAE או Accuracy
3 — מורכבות	ערך k או מורכבות אחרת	אותם מאפיינים וקיפולים	Train/Validation
4 — הוגנות	בדיקת קבוצות נפרדות	מדד וסף החלטה	שגיאות לפי קבוצות

יומן ניסויים קצר צריך לכלול תאריך, שם מאגר, מספר דוגמאות, מאפיינים, שיטת חלוקה, seed, היפר־פרמטרים, מדדים והסבר למה למדנו מהתוצאה. בפרויקט גמר דרישות התיעוד אינן קישוט: הן מאפשרות לשחזר את הניסוי ולהגן על החלטות מקצועיות.

10. מעבדת Python מודרכת — קו בסיס לעומת רגרסיה

להלן נתוני הוראה סינתטיים. הקוד מפריד ל־Train/Test, בונה קו בסיס של ממוצע ומודל ליניארי, ומחשב MAE על אותה קבוצת בדיקה. הספר מסביר את התהליך במלואו; במחברת התלמיד הנלווית יהיו ניסויים חדשים להשלים ללא פתרונות.

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.dummy import DummyRegressor
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error

X = np.arange(1, 21).reshape(-1, 1)
y = 12 + 4 * X.ravel() + np.array([
    0, 1, -2, 1, 0, -1, 2, -2, 1, 0,
    -1, 1, 2, -1, 0, 1, -2, 2, 0, -1
])
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)
baseline = DummyRegressor(strategy="mean")
model = LinearRegression()
baseline.fit(X_train, y_train)
model.fit(X_train, y_train)
for name, estimator in [("Baseline", baseline), ("Linear", model)]:
    prediction = estimator.predict(X_test)
    print(name, mean_absolute_error(y_test, prediction))
```

הסבר: np.arange יוצר ערכי מאפיין מ־1 עד 20; reshape הופך אותם למטריצה של עשרים שורות ועמודה אחת, כי scikit-learn מצפה שמאפיינים יוגשו במבנה דו־ממדי. מערך y מכיל ערך בקירוב ליניארי עם רעש קטן. שתי השיטות מותאמות רק על Train ונבחנות על אותם X_test ו־y_test. השוואת שתי התוצאות היא הוגנת מבחינת קבוצת הבדיקה — אך אם נבחר מודל בעקבותיה, אסור לדווח על אותו Test כאילו היה מבחן סופי עצמאי.

למה השתמשנו במאגר קטן ומומצא? כדי לבודד את הרעיון המתמטי ולבדוק תחילה שקוד הבסיס פועל. בניסוי אמיתי חייבים להבין כיצד הנתונים נאספו, האם דוגמאות עצמאיות זו מזו, מה מייצג כל מאפיין ומהן ההשלכות של טעות חיזוי.

11. מתי מודל מצליח מספרית אך לא מתאים לפרויקט?

נניח שמודל מסווג מחשבים פגומים ומחמיץ מעט תקלות חמורות, אך מסמן כמעט כל מחשב כתקול. Recall עשוי להיות גבוה, אך השירות לא יהיה שימושי. לחלופין, מודל יכול להצליח בבדיקה פנימית אך להיכשל בשבוע הבא בגלל החלפת חומרה, שינויים בסביבת העבודה או מדידות חדשות. לכן פיתוח מערכת AI כולל גם בדיקת איכות הנתונים, שיקולי שימוש, פרטיות ובקרה לאחר ההטמעה.

11.1 ארבע שאלות שמפתחים חייבים לשאול

- האם הנתונים מייצגים את מי או את מה שהמערכת תפגוש בעתיד?
- האם המאפיינים יהיו ידועים באמת בזמן החיזוי?
- מהי עלות טעות מהסוג החמור יותר, והאם המדד שבחרנו משקף אותה?
- האם אדם יוכל להבין מתי המודל אינו בטוח או מתי צריך להעביר החלטה לבדיקה אנושית?

12. קשר ישיר לפרק הבא: פרספטרון ורשתות עצביות

בפרקים הקודמים השתמשנו במסווגים שמנבאים על סמך קרבה של דוגמאות או מרכזי מחלקות. עכשיו אנחנו מוכנים להכיר מודל שמשתמש במשקולות: עבור וקטור מאפיינים x ומשקולות w, מחשבים סכום משוקלל ומוסיפים הטיה b. במסווג בינארי אפשר להעביר את התוצאה דרך כלל החלטה.

$$z = w \cdot x + b$$

הרעיון החדש אינו עצם המכפלה הפנימית, שכבר למדנו בפרק הווקטורים. הרעיון החדש הוא כיצד משנים משקולות בעקבות שגיאות. בפרספטרון נלמד כלל עדכון המיועד לסיווג; בהמשך נלמד אופטימיזציה המבוססת על נגזרות, ובפרקים על רשתות נרחיב את העיקרון למספר שכבות ול־Backpropagation. כך כל נושא חדש נשען על מושגים שכבר הבנו.

13. טעויות נפוצות ותיקונן

כיצד מתקנים?	טענה שגויה
הביצועים באימון אינם מוכיחים הכללה; דרושה הערכה חיצונית.	אם Accuracy באימון הוא 100%, המודל מצוין.
בחלק מהמודלים פרמטרים נלמדים מתוך נתונים; היפר־פרמטרים מכוונים בנפרד.	כל פרמטר במודל נבחר מראש.
שניהם מתארים שגיאה אך מענישים גדלים שונים באופן שונה.	MSE ו־MAE מודדים אותו הדבר.
שימוש ב־Test לבחירה פוגע בעצמאותה של ההערכה הסופית.	המודל שניצח על Test הוא הסופי.
מורכבות עשויה גם להתאים רעש ולפגוע בביצועים מחוץ לאימון.	ככל שמוסיפים מורכבות, המודל תמיד משתפר.
ייתכן ששגיאת ולידציה לא תרד או אף תעלה.	כשיש ירידה בשגיאת אימון, יש שיפור אמיתי.
התאמת התקנון חייבת להתבצע על Train בלבד.	אפשר לתקנן את כל המאגר ואז לחלק.
יש להתחשב גם בעלות, יציבות, הוגנות ודרישות המערכת.	שיפור מספרי קטן מצדיק שימוש במודל.

14. תרגול מדורג — ללא פתרונות

רמה א׳ — מושגים וחישובים

- א1. הסבירו בשתי פסקאות את ההבדל בין מודל לבין אלגוריתם אימון, ובחרו דוגמה מתוך רגרסיה.
- א2. עבור $y=(4,6,8)$ ותחזיות $\hat{y}=(5,5,8)$, חשבו את השאריות, MAE ו־MSE, וציינו את יחידות המדדים.
- א3. סווגו את המונחים הבאים לפרמטר, היפר־פרמטר או מאפיין: מספר שכנים, מספר התקלות, שיפוע רגרסיה, קצב למידה.
- א4. מה ההבדל בין תחזית של מודל לבין הערך האמיתי? מדוע איננו משתמשים בתווית האמיתית בזמן חיזוי של מקרה חדש?

רמה ב׳ — פרשנות, ניסויים וגרפים

- ב1. נתונות שתי עקומות: שגיאת אימון יורדת מ־1.2 ל־0.2, ושגיאת ולידציה יורדת תחילה ל־0.6 ואז עולה ל־0.9. הסבירו מה ייתכן שהתרחש.
- ב2. במאגר יש כמה רשומות לכל מחשב. הציעו חלוקה ל־Train ול־Test שמצמצמת זליגה.

• ב3. הציעו קו בסיס לבעיה של חיזוי זמן תיקון, והסבירו מדוע ממוצע האימון הוא בחירה סבירה להשוואה.

• ב4. מדוע אין לכוון עשרים היפר־פרמטרים על בסיס תוצאה חוזרת של אותה קבוצת Test?

• ב5. תכננו ניסוי המבודד את השפעת התקנון על KNN. אילו דברים חייבים להיות זהים בשתי ההרצות?

רמה ג׳ — תכנון מערכת וחשיבה ביקורתית

• ג1. מערכת חיזוי תקלות נבדקה על מחשבים משנת 2024, וכעת מותקנים מחשבים מדגם חדש. איזה סיכון להכללה נוצר? תכננו בדיקת ניטור.

• ג2. מודל A משיג Accuracy 0.90 אך מחמיץ יותר תקלות קשות ממודל B שמשיג 0.85. מה עוד צריך לדעת כדי לבחור?

• ג3. קוד מתקנן את כל X לפני train_test_split. זהו את הבעיה וכתבו בפסאודו־קוד את סדר הפעולות הנכון.

• ג4. בנו טבלת ניסויים של שלוש גרסאות מודל. כללו השערה, שינוי מבוקר, מדדים, מסקנה ומגבלה.

15. שאלת הערכה מקורית בסגנון בגרות

דודו בונה מערכת שמנבאת את משך התיקון של מחשב חדש. הוא בדק מודל ליניארי על שלוש תצפיות: הערכים האמיתיים הם (10, 12, 16), והתחזיות הן (9, 14, 15). המודל אומן על נתונים אחרים, אך במהלך בנייתו השתמש דודו בקבוצת הבדיקה כדי לבחור את קצב הלמידה.

• א. חשבו את שאריות החיזוי ואת MAE. הציגו את שלבי החישוב.

• ב. חשבו את MSE והסבירו מדוע הוא מעניש יותר שגיאה גדולה.

• ג. הסבירו מדוע הבחירה בקצב למידה בעזרת Test יוצרת בעיה בהערכת המודל.

• ד. הציעו תהליך חלוקה ובחירה מתוקן, וציינו מה ידווח בסוף.

• ה. ציינו סיכון אחד שעשוי להתעורר כאשר משתמשים במודל בבית ספר אחר.

הערה: זו שאלה מקורית שנכתבה עבור הספר לצורכי הכנה, ואינה מועתקת מבחינת בגרות רשמית. תשובות וקריטריונים לניקוד יופיעו בגרסת המורה.

16. מעבדת Colab — מהספר אל ניסוי עצמאי

מחברת התלמיד הנלווית מכילה 12 משימות: זיהוי מאפיינים ומטרות; השוואת תחזיות; חישוב MAE ו־MSE; שינוי משקל; ניסוי בקצב למידה; מודל קו בסיס; הפרדת Train/Test; בדיקת Leakage; השוואת מודלים; בניית יומן ניסויים; ניתוח עקומות; וסיכום להערכה בעל־פה. במקומות המיועדים לעבודה יופיע TODO או תא חסר. אין במחברת פתרונות מלאים למשימות.

התוצר הסופי של המעבדה אינו רק מחברת שרצה ללא שגיאות. על התלמיד להגיש שני גרפים, טבלה אחת של ניסויים ופסקה שבה הוא מסביר מדוע בחר במודל מסוים, מה הראיות לכך ומה אינו יכול להסיק מהנתונים.

17. מפת קשרים בין שלוש התכניות

שימוש עתידי	מה כבר נלמד ומה העומק כאן	מסלול
ניתוח בסיסי והסבר אלגוריתמים	היכרות עם אימון, חיזוי, מדדים והכללה	יחידה 3 — כיתה י׳
הצדקת תהליך מדעי והכנה	פונקציות מרחק, מדדי הצלחה,	נושאי ליבה — יחידה 5

שימוש עתידי	מה כבר נלמד ומה העומק כאן	מסלול
לבגרות	ולידציה וחקר היפר־פרמטרים	
פרספטרון, Gradient Descent, רשתות ופרויקט גמר	תהליך האימון כמערכת של פרמטרים, שגיאה, בחירה ובדיקה	התמחות למידה עמוקה — פרק 8

עוגן התכנית: במפת ההתמחות שבמאגר GitHub מופיע פרק 8 ״מושגי יסוד בלמידת מכונה״ בהיקף 10 שעות, בין KNN לפרספטרון. סדר תתי־הנושאים בפרק זה הוא עיבוד דידקטי שלנו, ולא העתק מאומת של תתי־סעיפי ה־PDF הרשמי.

18. הצעת חלוקת זמן להוראה — 10 שעות

שעות	מוקד	שיעור
2	מודל, נתונים, תחזית, פרמטר והיפר־פרמטר	1
2	שאריות, MAE, MSE, קו בסיס וחישוב ידני	2
2	אימון, שינוי פרמטרים, העמקת Overfit/Underfit וקריאת עקומות	3
2	הכללה, זליגת מידע, ניסוי מבוקר ופרשנות	4
2	מעבדת Python, שאלת הערכה, יומן ניסויים והכנה לפרספטרון	5

זוהי חלוקת שעות מוצעת למורה (שעה לימודית = 45 דקות), לא חלוקה רשמית שהועתקה מהסילבוס.

19. מילון מושגים

פירוש קצר	מונח
פונקציה המפיקה תחזית מקלט	Model
תהליך התאמת פרמטרים מנתוני אימון	Training
ערך שהמודל מתאים בהתאם לנתונים	Parameter
הגדרה המכוונת את תהליך האימון או צורת המודל	Hyperparameter
פונקציה המודדת את הפער בין תחזית לתוצאה	Loss
מעבר אחד על נתוני האימון, לפי שיטת האימון	Epoch
יכולת לתפקד היטב על נתונים חדשים	Generalization
התאמת יתר שעלולה לפגוע בהכללה	Overfitting
כלל פשוט המשמש אמת מידה להשוואה	Baseline
שימוש במידע שלא אמור להיות זמין בעת האימון או החיזוי	Data Leakage
יכולת לשחזר ניסוי בתנאים מתועדים	Reproducibility

20. חקר מקרה מסכם — האם המודל החדש באמת עדיף?

דודו אסף שישה מחשבים ותיעד עבור כל אחד מהם את מספר התקלות ואת זמן התיקון בפועל. כדי להמחיש את הבדיקה בלי להסתבך במאגר גדול, נתוני האימון הם ארבעת המחשבים הראשונים, ונתוני

הבדיקה הם שני מחשבים חדשים. הערכים פשוטים ומומצאים לצורך המחשה. בניסוי אמיתי אסור לבחור חלוקה כזו רק מפני שהיא גורמת למודל להצליח.

קבוצה	מספר תקלות x	זמן תיקון בפועל y
Train	1	20
Train	2	24
Train	3	28
Train	4	32
Test	5	34
Test	6	38

20.1 קו הבסיס — תחזית אחת לכולם

נגדיר Baseline שמנבא תמיד את ממוצע ערכי המטרה של האימון. שימו לב שאיננו משתמשים כלל בשני נתוני Test לחישוב הממוצע. הם שמורים לבדיקת הכללה.

mean(Train) = (20 + 24 + 28 + 32) / 4 = 26

לכן שתי תחזיות קו הבסיס הן 26. עבור המחשב הראשון ב־Test, שהזמן האמיתי שלו הוא 34, הטעות המוחלטת היא 8. עבור המחשב השני, שהזמן האמיתי שלו הוא 38, הטעות היא 12. ממוצע הטעויות המוחלטות הוא:

MAE_baseline = (8 + 12) / 2 = 10

20.2 מודל שלומד את הקשר בנתוני Train

כעת נסתכל על ארבע דוגמאות האימון בלבד. כל תוספת של תקלה אחת מלווה בעלייה של ארבע יחידות זמן. ישר שעובר בכל ארבע הנקודות מקיים את המשוואה הבאה:

ŷ = 4x + 16

בבדיקה החיצונית המודל מנבא 36 למחשב עם חמש תקלות ו־40 למחשב עם שש תקלות. הערכים האמיתיים הם 34 ו־38. לכן בכל אחת משתי הדוגמאות הטעות המוחלטת היא 2:

MAE_model = (2 + 2) / 2 = 2

המודל הצליח יותר מקו הבסיס על שתי דוגמאות הבדיקה האלה. זהו ממצא, אך לא הוכחה שהוא תמיד יצליח. המדגם קטן מאוד, שתי נקודות ה־Test הן מחוץ לטווח ערכי x של Train — כלומר יש כאן גם אקסטרפולציה — והקשר במחשבים אחרים עלול להיות שונה. ההבחנה בין ממצא לבין מסקנה כללית היא חלק מהחשיבה המדעית שאנחנו רוצים לפתח.

20.3 שלוש שאלות של בוחן פרויקט גמר

- מדוע אינך יכול לקבוע שהמודל שלך מדויק תמיד, אף ש־MAE שלו קטן?
- מה היה קורה לו היית משתמש גם בשני מחשבי Test לצורך בחירת השיפוע?
- איזה נתון אמיתי היית אוסף כדי לבדוק אם מספר התקלות מספיק לחיזוי משך התיקון?

21. דף תכנון ניסוי לתלמיד

לקראת מיני־פרויקט או פרויקט גמר, כל תלמיד יוכל להשתמש בטבלה הבאה כדי לתכנן ניסוי. לפני שרצים לקוד, ממלאים את העמודה הראשונה והשלישית. לאחר ההרצה ממלאים את התוצאות והפירוש. כך שומרים על ניסוי שניתן לשחזר ועל שיקול דעת שקוף.

מה אכתוב ביומן הניסוי?	השאלה
למשל: האם StandardScaler משפר את KNN על הנתונים שלי?	מהי מטרת הניסוי?
מקור המאגר, מספר רשומות, מאפיינים, דרך איסוף ורישוי	מהם הנתונים?
קבוצת Test, אותם קיפולים, אותו מדד ואותם מאפיינים	מה קבוע?
גורם יחיד — למשל תקנון עם/בלי	מה אני משנה?
מדד איכות ובמידת הצורך זמן חישוב או עלות	מה אני מודד?
מסקנה מוגבלת לנתונים שנבדקו והסבר חלופי אפשרי	מה למדתי?
בדיקה נוספת, איסוף נתונים, שיפור או דחיית הרעיון	מה הצעד הבא?

דוגמה לניסוח מסקנה מקצועית: ״בשלושה קיפולי ולידציה על נתוני האימון, התקנון שיפר את ממוצע הדיוק של KNN. לכן בחרנו להמשיך לבדוק מודל עם תקנון. עם זאת, איננו יודעים עדיין אם השיפור יישמר עבור נתונים ממחשבים מדגם חדש, ונידרש לבחון זאת במועד מאוחר יותר.״

דוגמה לניסוח שאינו מקצועי: ״התקנון תמיד משפר KNN, ולכן כל המודלים צריכים תקנון״. משפט כזה הופך תוצאה מקומית לכלל אוניברסלי בלי נתונים שמצדיקים זאת.

22. סיכום מושגי — מה חשוב לקחת לפרק הפרספטרון?

בפרק זה ראינו שמודל מפיק תחזית באמצעות פרמטרים, שתהליך האימון מתאים את הפרמטרים לנתונים, ושפונקציית השגיאה מאפשרת להגדיר מה נחשב שיפור. למדנו ששיפור על נתוני האימון אינו מספיק, ולכן מפרידים בין אימון, בחירה והערכה סופית. בדקנו כיצד זליגת מידע, חלוקה לא נכונה וקנה מידה עלולים לגרום לנו להעריך מודל באופן מוטעה.

בפרק הבא נפגוש את הפרספטרון. במקום לזכור רק נוסחת סיווג חדשה, נחפש בו את אותם מרכיבים: מהו וקטור הקלט? מהם המשקולות וההטיה? מהי התחזית? מתי מתרחשת טעות? כיצד מתקנים את המשקולות? באילו נתונים בודקים שהשינוי הועיל? הקישור בין השאלות הללו הופך את הלמידה ממאגר פקודות להבנה של מערכת לומדת.

מקורות מקצועיים ועוגני קריאה

עוגן תכנית: משרד החינוך — התמחות בלמידת מכונה ולמידה עמוקה, פרק 8 (10 שעות), על פי מפת התכנית השמורה במאגר Data-Science-ML-Course; נדרש אימות סופי של תתי־הסעיפים מול PDF הרשמי המעודכן.

מקורות מרכזיים להשראה מקצועית: Luis Serrano, Grokking Machine Learning; Aurélien Géron, Hands-On Machine Learning; James et al., An Introduction to Statistical Learning with Applications in Python; Deisenroth et al., Mathematics for Machine Learning; Goodfellow, Bengio & Courville, Deep Learning. ההסברים והדוגמאות כאן מקוריים ולא הועתקו מהספרים.

לבחינת המימושים והטעויות: scikit-learn User Guide (Common Pitfalls); Google Machine Learning Crash Course; מאגר הקורס: github.com/samiabo75/Data-Science-ML-Course.